Working with Images

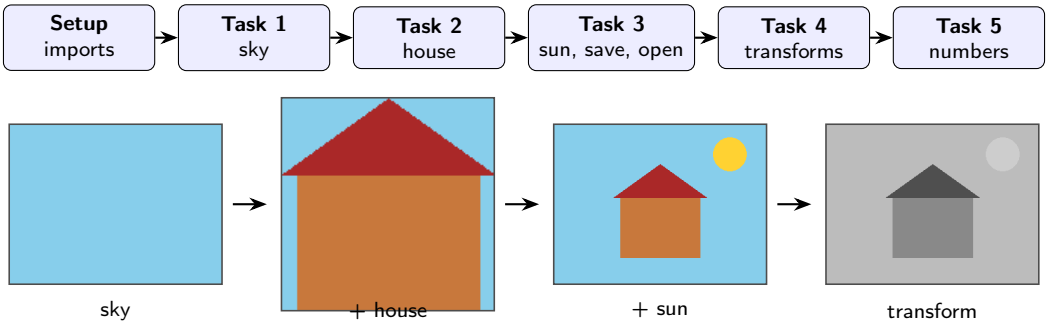
Lab Session: Build a picture, read its pixels, transform it

Mr. Jayakumar T V

Department of Avionics
Indian Institute of Space Science and Technology

Duration: 45 min

Roadmap: build one small picture, step by step



Each task is **one short snippet**: type it, run it, and compare with the expected output. After each step we **read** a pixel to confirm what is in the picture.

Setup: import what we need

```
from PIL import Image, ImageDraw
import numpy as np
from IPython.display import display
```

Command	Purpose
`Image`	create, open, save and transform pictures
`ImageDraw`	draw shapes on a picture
`numpy as np`	turn a picture into numbers (Task 5)
`display(img)`	**show** the picture under the cell

If an import fails

Run `pip install pillow numpy` once, then restart the notebook.

Seeing pictures in Jupyter

`print(img)` only prints text. Use `display(img)`, or put `img` alone on the *last* line of a cell.

Coordinates

A point is `(x, y)`. $(0, 0)$ is the **top-left**; x goes right, y goes *down*.

Task 1: a blank canvas is the sky

```
img = Image.new("RGB", (320, 240), (135, 206, 235))
print(img.size, img.mode)
print("sky  :", img.getpixel((10, 10)))
display(img)
```

```
(320, 240) RGB
sky  : (135, 206, 235)
[sky-blue picture shown]
```

Command	Purpose
`Image.new(mode, size, colour)`	a new picture, all one colour
`img.size`	**(width, height)**
`img.getpixel((x, y))`	read one pixel (R, G, B)



The yellow dot is the pixel we read.

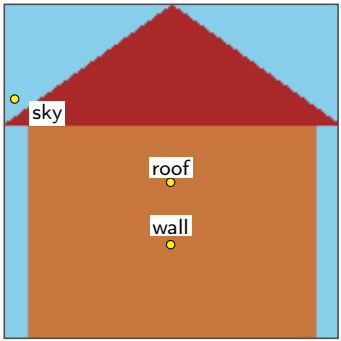
Your turn

Make the canvas 400 × 300. What does `img.size` print?

Task 2: add a house, then read the wall, the roof and the sky

```
d = ImageDraw.Draw(img)
d.rectangle((100, 110, 220, 200), fill=(200, 120, 60))
d.polygon([(90, 110), (160, 60), (230, 110)],
          fill=(170, 40, 40))
print("wall :", img.getpixel((160, 150)))
print("roof :", img.getpixel((160, 90)))
print("sky  :", img.getpixel((10, 10)))
display(img)
```

```
wall : (200, 120, 60)
roof : (170, 40, 40)
sky  : (135, 206, 235)
[picture with house]
```



`ImageDraw.Draw(img)`	a pen: drawing changes `img`
`d.rectangle(box, fill)`	box = (left, top, right, bottom)
`d.polygon(points, fill)`	shape through the corner points

Your turn

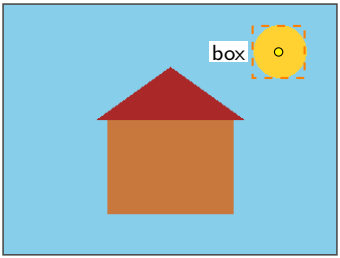
Predict `getpixel((160, 70))` and `getpixel((160, 55))`. Then run them.

Task 3: add the sun, save the picture, open it again

```
d.ellipse((240, 20, 290, 70), fill=(255, 210, 50))
print("sun  :", img.getpixel((265, 45)))

img.save("scene.png")
img = Image.open("scene.png")
print(img.format, img.size, img.mode)
display(img)
```

```
sun  : (255, 210, 50)
PNG (320, 240) RGB
[finished scene shown]
```



The dashed box is the ellipse bounding box.

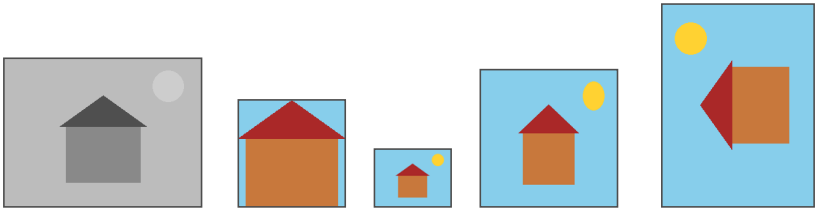
`d.ellipse(box, fill)`	circle if the box is square
`img.save(name)`	writes a file; `.png` picks the format
`Image.open(name)`	reads it back; `img.format` is now `PNG`

Your turn

Which pixel is the box corner `(240, 20)`: sun or sky? Check with `getpixel`.

Task 4: five simple transforms of the scene

```
gray  = img.convert("L")              # one number per pixel
house = img.crop((90, 60, 230, 200))  # cut out the house
small = img.copy(); small.thumbnail((100, 100))
sq    = img.resize((224, 224))
turn  = img.rotate(90, expand=True)
display(gray, house, small, sq, turn)
```



gray
mode L

house
140 × 140

small
100 × 75

sq
224 × 224

turn
240 × 320

`display` accepts several pictures at once. Every transform returns a **new** picture and leaves `img` unchanged. The only exception is `thumbnail`, which edits in place, so we copy first.

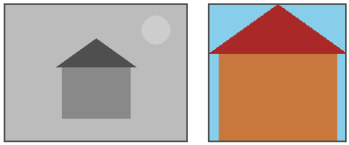
Task 4a: convert("L") and crop

```
gray = img.convert("L")
print(gray.mode, gray.getpixel((265, 45)))
print(gray.getpixel((10, 10)), gray.getpixel((160, 150)))

house = img.crop((90, 60, 230, 200))
print(house.size)
display(gray, house)
```

```
L 205
188 137
(140, 140)
[gray scene, house crop]
```

`convert("L")`	gray: $0.299R + 0.587G + 0.114B$
`crop((l, t, r, b))`	cut a rectangle; size $= (r - l, b - t)$



gray crop

Sun: $76.2 + 123.3 + 5.7 = 205$
Wall: $59.8 + 70.4 + 6.8 = 137$

Your turn

What is the cropped size for `img.crop((100, 110, 220, 200))`, the wall only?

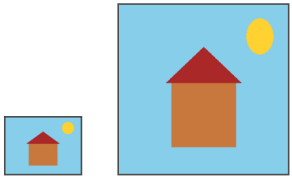
Task 4b: `thumbnail` and `resize`

```
small = img.copy()
small.thumbnail((100, 100))
print(small.size, img.size)

sq = img.resize((224, 224))
print(sq.size)
display(small, sq)
```

```
(100, 75) (320, 240)
(224, 224)
[small picture, square picture]
```

`img.copy()`	duplicate, so the original stays safe
`thumbnail((w, h))`	shrink to fit, **keep proportions**, in place
`resize((w, h))`	exactly this size, may stretch



100×75 224×224

Scale $= \min(\frac{100}{320}, \frac{100}{240}) = 0.3125$, so 100×75.

Your turn

What is `small.size` after `small.thumbnail((50, 50))` on a fresh copy?

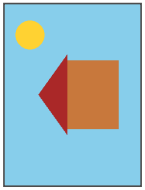
Task 4c: rotate and transpose

```
turn = img.rotate(90, expand=True)
print(turn.size)

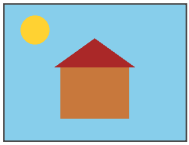
mir = img.transpose(Image.FLIP_LEFT_RIGHT)
print(mir.size, mir.getpixel((54, 45)))
display(turn, mir)
```

```
(240, 320)
(320, 240) (255, 210, 50)
[turned picture, mirrored picture]
```

`rotate(90, expand=True)`	turn counter-clockwise; canvas grows, width and height swap
`transpose(mode)`	`FLIP_LEFT_RIGHT` mirrors: $x \mapsto 319 - x$, size same



240 × 320



mirror

The sun was at $x = 265$; the mirror puts it at $319 - 265 = 54$.

Your turn

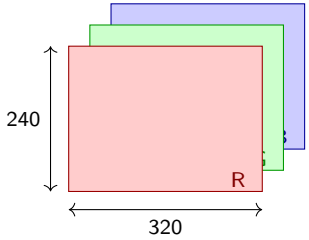
What is `img.rotate(90).size` *without* `expand=True`?

Task 5: the picture as a NumPy array

```
a = np.array(img)
print(a.shape, a.size)
print(a[45, 265])
print(a[120:190, 110:210].mean(axis=(0, 1)))
```

```
(240, 320, 3) 230400
[255 210  50]
[200. 120.  60.]
```

`np.array(img)`	pixels as numbers
`a.shape`	(height, width, 3)
`a[y, x]`	row first! `getpixel` is `(x, y)`
`a[y1:y2, x1:x2]`	a region; `.mean` of the wall = wall colour



$240 \times 320 \times 3 = 230{,}400$ numbers

Your turn

What does `img.getpixel((45, 265))` do, and why?

Your-turn answers

Task	Answer	Why
1	`(400, 300)`	`size` is whatever `Image.new` was given
2	`(170, 40, 40)` then `(135, 206, 235)`	at $y = 70$ the roof spans $x \in [146, 174]$; the apex is at $y = 60$, so $y = 55$ is sky
3	sky `(135, 206, 235)`	the circle touches the box edges, not its corners
4a	`(120, 90)`	$220 - 100$ and $200 - 110$
4b	`(50, 38)`	scale 0.15625; height 37.5 is rounded to 38
4c	`(320, 240)`	without `expand=True` the canvas keeps its size and the corners are cut
5	`IndexError`	$y = 265$ is outside the 240 rows; `(x, y)` was given as `(45, 265)`, the reverse of `a[45, 265]`

Recap: commands used today

Task	Commands	Purpose
Setup	`import Image, ImageDraw, numpy`	tools
1	`Image.new`, `size`, `mode`, `getpixel`	create, read
2	`ImageDraw.Draw`, `rectangle`, `polygon`	draw
3	`ellipse`, `save`, `Image.open`, `format`, `display`	finish, save, show
4	`convert`, `crop`, `copy`, `thumbnail`, `resize`, `rotate`, `transpose`	change
5	`np.array`, `shape`, `a[y, x]`, `mean`	numbers

Common mistakes

`size` is (width, height) but `shape` is (height, width, 3). `thumbnail` changes the picture it is called on. Rotating without `expand=True` cuts the corners.

Questions?